Estimaciones: tiene que ver con la predicción acerca de algo que podría pasar en el futuro. La dificultad de esto es el nivel de incertidumbre con el que trabajamos para definir esa predicción, para esto debemos buscar métodos y técnicas para que esas predicciones sean lo más efectivas posibles.

- **Consideraciones**:
  - ★ Las estimaciones NO son precisas.
  - ★ Estimar no es planear y planear no es estimar.
  - ★ Las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
  - ★ Mientras mayor sea la diferencia entre lo estimado y lo planeado, aumenta el riesgo.
  - ★ Las estimaciones no son compromisos.

El nivel de incertidumbre en las estimaciones va disminuyendo a medida que el proyecto avanza, pero al ocurrir eso el proceso de estimaciones es algo que hacemos a lo largo de toda la vida del proyecto y tomando decisiones en función de eso.

- **Posibles errores de estimación**: pueden tener múltiples orígenes, por ejemplo utilizar alguna técnica incorrecta, no ir ajustando en función de los valores reales que tengo, utilizar un único método de estimación.
- **Técnicas fundamentales de estimación**:
  - ★ Contar: requerimientos, personas involucradas, funcionalidades, horas de trabajo, cantidad de días, etc.
- **Métodos utilizados**:
  - ★ Basados en la experiencia. Particularmente veremos esta.
    - *Métodos*: suele ser de mucha utilidad combinarlos. Al tratarse de una medida con alta incertidumbre, se suele estimar con rango o porcentajes de probabilidad de ocurrencia, no de un único valor absoluto.
      - **Datos históricos**: tiene que ver con utilizar información existente para poder realizar estimaciones relacionadas con un nuevo proyecto.
        ¿Dificultades?
        - No siempre la experiencia es extrapolable, y aunque lo fuera, necesito poder almacenar esos datos históricos en una base de conocimiento que de alguna forma me permita tipificar esos datos para que me sirvan.
        - ¿Qué datos históricos necesito?
      - **Juicio experto**.
        - Puro: tengo a un experto que sabe del tema, que estudia las especificaciones del proyecto y a partir de esto realiza la estimación.
          Enfoque más utilizado en la práctica.

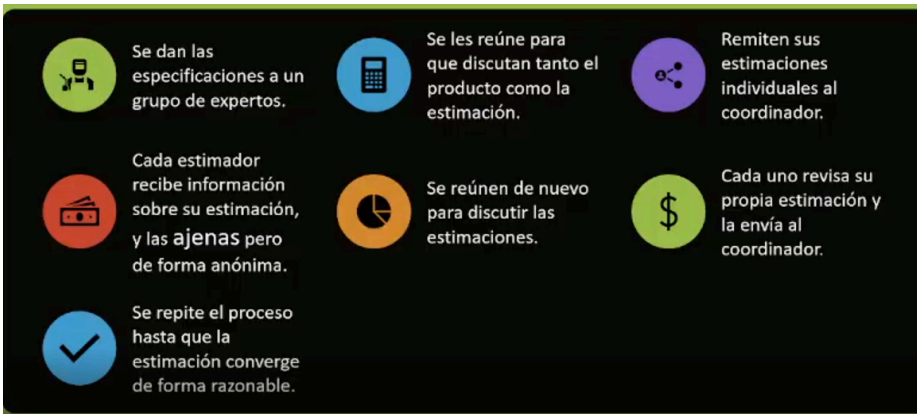
Estructurando el juicio de experto:

- Tenga tareas de granularidad aceptable.
- Use el método de “optimista, pesimista y habitual” y su fórmula = (o+4h+p)/6 (CUIDADO!, buen comienzo pero pésimo final).
- Use un checklist y un criterio definido para asegurar cobertura

¿Dificultades?

- Si el experto desaparece, la empresa deja de estimar.
- Subjetividad por parte del experto.
- Que el experto considere que tiene ciertos derechos que en realidad no y por eso estime mal.

- Delphi: un grupo de personas son informadas y tratan de adivinar lo que costará el desarrollo tanto en esfuerzo, como en duración. Las estimaciones en grupo suelen ser mejores que las individuales. El secreto es que la estimación se hace en términos de consenso:

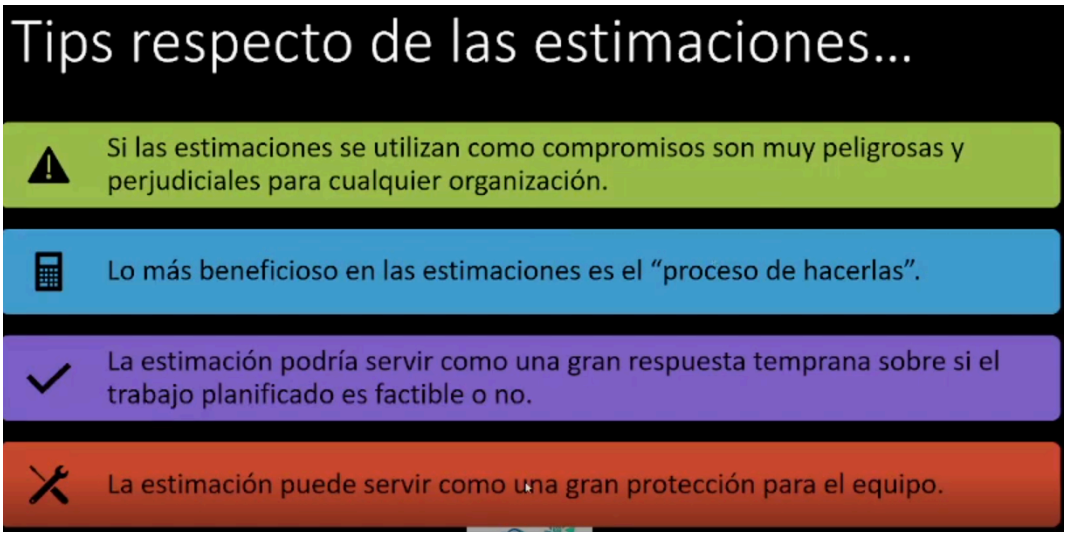


En las estimaciones tradicionales estimamos tamaño del producto a construir, esfuerzo que nos requerirá una determinada tarea, costos, tiempo calendario, recursos. Utilizamos una unidad de medida absoluta.

- **Analogía**.

★ Basados exclusivamente en los recursos.
★ Método basado exclusivamente en el mercado.
★ Basados en los componentes del producto o en el proceso de desarrollo.
★ Métodos algorítmicos.

Las estimaciones tradicionales y las ágiles tienen una misma base:



Estimaciones ágiles: a diferencia de las estimaciones tradicionales, las features/stories son estimadas usando una unidad de medida de tamaño relativo (o sea no absoluta) conocida como story points (SP). Este tipo de unidad de medida solo es válida para las personas que participan del proyecto y no está basada en el tiempo, sino más bien con el tamaño.

Estimación relativa:

- ★ Las personas no saben estimar en términos absolutos.
- ★ Somos buenos **comparando** cosas.
- ★ **Comparar** es generalmente más rápido.
- ★ Se obtiene una mejor dinámica **grupal** y pensamiento de equipo más que individual.
- ★ Se emplea mejor el tiempo de análisis de las stories.
- ★ ¿Y el tamaño?
  - "La palabra tamaño refiere a cuán grande o pequeño es algo".
  - El tamaño es una medida de la cantidad de trabajo necesaria para producir una feature/story.
  - El tamaño indica:
    - Cuán compleja es una feature/story.
    - Cuánto trabajo es requerido para hacer o completar una feature/story.
    - Y cuán grande es una feature/story.

Descripción del Trabajo vs Esfuerzo:

- ★ Las estimaciones basadas en tiempo son más propensas a errores debido a que pueden variar por muchas razones como:
  - Habilidades.
  - Conocimiento.
  - Asunciones.
  - Experiencia.

- Familiaridad con los dominios de aplicación/negocio.

★ Tamaño NO ES esfuerzo.

★ ¿Qué hacemos con el tamaño?
- **Tamaño por números**: 1 a 10.
- **Talles de remeras**: S, M, L, XL, XXL.
- **Serie 2^n**: 1, 2, 4, 8, 16, 32, 64, etc.
- **Fibonacci**: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.
  Muy útil para medir user stories ya que éstas crecen de manera exponencial, no lineal.

Una vez elegida la escala no se cambia ya que de hacerlo, cambiamos el metro patrón.

★ ¿Qué es un Story Point?
- Es una unidad de medida RELATIVA específica (del equipo) de, complejidad, incertidumbre/riesgo y esfuerzo, es lo que “el kilo” a la unidad de nuestro sistema de medición de peso.
- Story point da idea del “peso” de cada story y decide cuán grande (compleja) es.
- La complejidad de una feature/story tiende a incrementarse exponencialmente.

Poker estimation: variante/mejora del método juicio experto delphi.

★ Popular entre los Agile practicioners.

★ Combina opinión de experto, analogía y desegregación.

★ Participantes en “planning poker” son desarrolladores.
- “Las personas más competentes en resolver una tarea deben ser quienes la estiman”.

¿Cómo “decodificar” las estimaciones?



Cuando hagamos la estimación le asignaremos a las user stories una medida relativa, en términos de story point, y que es un peso que le asignaremos a la misma en función de complejidad, incertidumbre y esfuerzo.
Primero elegiremos una historia pequeña, preferentemente una funcionalidad a la que podamos asignarle un peso de un story point. A partir de esto ya tenemos una base para iniciar la comparación. Esta base se denomina “user story canónica”.

Poker planning:

- ★ Prerrequisitos:
  - Lista de features/stories a ser estimadas.
  - Cada estimador tiene un mazo de cartas.
- ★ Pasos:
  1. Determine la base story (la canónica) que será usada para comparar con las otras stories.
     a. La story a ser estimada se lee a todo el equipo.
     b. Los estimadores discuten la story, haciendo preguntas al product owner (las que se necesiten).
     c. Cada estimador selecciona una carta y la coloca boca abajo en la mesa.
     d. Cuando están todas las cartas sobre la mesa, las mismas se exponen al mismo tiempo.
     e. Si todos los estimadores seleccionan el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos A partir de esto, se vuelve a estimar. Este proceso se repite n veces hasta que el grupo converge en un único valor.

  2. Se toma la próxima story, se discute con el product owner.
  3. Cada estimador asigna a la story un valor por comparación contra la base story. “Cuan grande/pequeña, 23 compleja, riesgosa es esta story comparada con Story Z?”.

Agile es empírico, inspeccionar y adaptar es mandatorio.

Gestión de productos de software: cuando vamos a crear un nuevo producto de software, esa creación tiene una determinada línea de crecimiento.

1. Validación de hipótesis.
2. Definición de alcance.
3. Diseño y desarrollo.
4. Implementación.
5. Crecimiento y lealtad.
6. Experiencia del usuario.

El mayor riesgo no es fallar técnicamente, es construir a la perfección algo que nadie quiere (funcionalidades que no se utilizan).
El pensamiento Lean define la creación de valor como proveer beneficios reales a los clientes. Cualquier otra cosa es desperdicio.
La productividad no se mide en líneas de código, sino en averiguar sistemáticamente cuál es la cosa correcta a construir.

Escribir código no es sinónimo de crear valor. El éxito no se mide por la cantidad de funcionalidades entregadas, sino por la adopción real del cliente.

La brecha evolutiva del ingeniero de software:

- ★ Focalizado en tareas: crear un producto de software que resuelva el problema que queremos resolver.
- ★ Focalizado en experiencias: que ese producto de software le proporcione al usuario una experiencia amigable y que genere que quiera utilizar ese software y no otro.

Producto Mínimo Viable: es un producto que a partir de una propuesta concreta de un software que podría ser aceptado en el mercado, para validar esta hipótesis, voy a construir un producto pequeño y confirmarlo. Este producto se hace con el menor de los esfuerzos. Sin embargo, para validar esa hipótesis, necesitamos que nuestro usuario perciba el software como si este funcionara.
Nos tenemos que asegurar de no demorarnos para no perder la oportunidad.

- **La audacia de cero**: el número cero invita a la pura imaginación de los inversores. Esto crea un incentivo brutal para retrasar el lanzamiento hasta que el producto sea “perfecto”.
- **El costo del retraso**: posponer el MVP aumenta el esfuerzo desperdiciado, pierde feedback temprano esencial y maximiza el riesgo de construir algo que nadie quiere.

UVP (Propuesta de Valor Única): la hipótesis principal. Lo que hace que el producto o servicio sea único en el mercado. A partir de esto creo el MVP que será el vehículo para probar la hipótesis. Debe incluir funciones básicas para ser genuinamente “viable”. Luego, tengo que explorar si este MVP es aceptado y qué características señalan los usuarios que podríamos agregarle. Si todo el mundo identifica cosas diferentes, la hipótesis no se valida.

Pivote: normalmente vamos a tener una UVP y a partir de ella nosotros proponemos un MVP. Si el mercado no nos permite validar la hipótesis por el motivo que sea, ese MVP queda descartado y tendremos que construir uno nuevo, lo que no representa un problema ya que al primero lo construí con el mínimo esfuerzo.

Suponiendo que ya llegamos al MVP que agrega valor al mercado, ya tenemos que empezar a pensar en el producto mínimo comercializable. En simultáneo, también puedo ir trabajando con características mínimas viables (dentro del producto que ya empieza a existir, puedo seguir haciendo pruebas de valor con estas pequeñas características).

Aunque los MVP, MMF y MVF son atómicos desde la perspectiva de negocio (no se puede aprender de algo más pequeño), son gigantescos desde la perspectiva de desarrollo. “Mínimo” no está relacionado al tamaño, sino a la base necesaria para desarrollar algo funcional.

MLP (Mínimo Producto Adorable): no me alcanza con que la idea sea de un producto que lo van a usar; teniendo en cuenta que en el mercado hay otros competidores, aún cuando hubiera otro producto de características similares, quiero que mis usuarios elijan mi producto por sobre el resto.
¿Cómo lograrlo? reputación de la marca, el producto de software logra generar una conexión temprana con los usuarios, exploré de mejor manera la adopción inicial de ciertos usuarios.

MVP (Producto Mínimo Viable)	**MMP** (Producto Mínimo Comercializable)	**MLP** (Producto Mínimo Adorable)
Enfoque: Entender necesidades Validar conceptos rápido y barato.	**Enfoque:** Amplio alcance / Atender a grandes bases de usuarios.	**Enfoque:** Diseñar para el deleite / Conexión emocional.
Foco Clave: Investigación, Código base, Pruebas de usuario.	**Foco Clave:** Estabilidad técnica, Lanzamiento al mercado.	**Foco Clave:** UI avanzada, Impacto emocional, Reducción de fricción.
Resultado: Informar el alcance del producto.	**Resultado:** Atracción de mercado masivo.	**Resultado:** Lealtad temprana y retención.